

API CONTRACT

SCHEMA v2

Grid Response Contract

A backend-driven grid contract. The backend decides **what** to show; the UI owns **how** it's drawn.

Blocks

10

Capability
families

4

Grid engine

AG Grid

Wire format

**JSON ↔
proto**

1 Purpose
2 Top-level
structure

3 Section
reference
4 Capability
catalog

5 Color &
theming
6 Registry &
handlers

7 Worked
example
8 Proto mapping

1 Purpose & design philosophy

The response is a **backend-driven grid contract**. Functions can't be serialized, so they never travel on the wire. They are **string references** that the UI resolves against a UI-owned **registry**.

BACKEND DECIDES — WHAT

- Data values
- Structure: headers, pinned rows, detail
- Which column uses which capability
- Saved sort / filter state

UI OWNS — HOW

- Renderers, editors, tooltips
- Click / action handlers
- Default colors & styling tokens
- The registry of legal names

Rule: `"handler": "onRowAction"`

→ the UI looks it up in the registry → real function.

2 Top-level structure

The response has ten blocks. The metadata blocks are **maps keyed by** field or row id, so lookups are direct and sparse.

BLOCK	KEYED BY	HOLDS
 schemaVersion	—	Contract version string
 gridMetaData	—	Grid-wide AG Grid options + grid-level heatmap
 columnDefs	—	Header layout only (nested children), no styling
 columnMetadata	<code>field</code>	Per-column typed styling/behaviour + saved sort/filter

 rowData	—	Raw values only — keyed row objects with id
 rowMetadata	 id	Per-row gating, action resolution, total-row styling
 cellMetadata	 id → field	Per-cell overrides (tooltip, raw-hex color)
 detailData	 id	Master/detail child rows (or lazy fetch)
 pinnedTopRows	—	Pinned rows at the top (N rows)
 pinnedBottomRows	—	Pinned rows at the bottom (N rows) — totals, limits

3

Section-by-section config reference

gridMetadata grid-wide options

defaultColDef { sortable, filter, resizable, floatingFilter }

pagination

paginationPageSize

rowSelection

getRowId: "id"

masterDetail

pivotMode

heatmap { scope:"grid", min, max, centerline, positiveColor, negativeColor }

The heatmap is grid-level: one scale across the grid, with no columns list. Columns opt out via columnMetadata.

columnDefs header layout

headerName field pinned width

colType: StringType | DoubleType | CustomType children

colId and groupHeaderName are dropped: field is the key, and AG Grid derives column groups natively from nested children.

columnMetadata[field] per-column capability

type: heatmap | bar | badge | link | action | ...

<typed config> sort + sortIndex filter + filterParams

editable / validations actions

type selects a capability and its matching typed config. sort and filter hold the saved (Pattern A) state that is restored on load.

rowData raw values

id <value fields> <attribute fields>

Raw cell values and attribute fields only. No styling and no flags, so the same rows can feed both charts and grids.

rowMetadata[id] per-row

actions { <id>: { enabled, visible, reason } } selectable

selected rowType: GRAND_TOTAL | SUB_TOTAL bold bgColor

excludeHeatMap error

Per-row action gating (hidden by default), selection, and total-row styling, such as Global / AMER / EMEA totals excluded from the heatmap.

cellMetadata[id][field] per-cell

tooltip color bgColor icon

One-off overrides only. Use it sparingly: anything repeating belongs in columnMetadata or rowMetadata.

4 Capability catalog — the four families

Every column or grid feature belongs to one family. The family's type picks its config shape; anything left out falls back to the defaults below.

Tooltip

columnMetadata

tooltip

```
tooltip: { renderer:  
  "customTooltip" } ·  
cellMetadata tooltip overrides  
text
```

Styling

columnMetadata

heatmap

participates in grid heatmap ·
default on for numbers

bar (diverging)

type: "bar" · centerline from
heatmap · auto max

badge

```
badge: { static: true, value:  
  "$MM" }
```

posNeg

negative in () + red · default
on

Interaction

columnMetadata ·
rowMetadata

Structure

gridMetaData · blocks

link

handler: "<registry name>" .
params

action column

action.items[] { id, icon,
handler } . enabled per row in
rowMetadata

clickable

default false . total rows
never clickable

master-detail

masterDetail . detailData[id]
or detailEndpoint

pivot

pivotMode . pivot / value
fields

tree

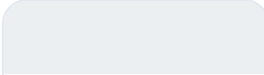
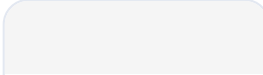
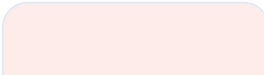
treeData . treePathField

pinned rows

pinnedTopRows /
pinnedBottomRows . N rows

5 Color & theming rules

The **UI owns the default token** → **hex table**. The server sends **raw hex only to override**, either on the specific config or in cellMetadata. There is **no palette block** on the wire.

 positive #2e7d32	 negative #c62828	 grandTotalBg #eceff1	 subTotalBg #f5f5f5
 breachBg #ffebee	 warning #e0a030	 errorRow #fdecea	 link #2f62c9

Resolution order:

UI default token → config hex →

rowMetadata bgColor →

cellMetadata hex (wins)

6 Registry & handlers

The registry is a UI-owned list of the legal renderer, handler, editor and tooltip names. The wire carries **only string references**, and an unknown name falls back to the default with a console warning. Every handler receives the same **context object**.

RENDERERS

heatmap bar badge link
action limitRange

HANDLERS

onRowAction onCellClick
openDetail

EDITORS

number text select date

TOOLTIPS

customTooltip

HANDLER CONTEXT

```
handler(ctx) {  
  ctx.actionId // "edit" |  
  "delete" | "copy"  
  ctx.rowId // "row_2"  
  ctx.field // "actions"  
  ctx.value // cell  
  value  
  ctx.row // full  
  rowData object  
  ctx.rowMeta //  
  rowMetadata[rowId]  
  ctx.params // from  
  config  
  ctx.emit(event) // onEvent  
  to consumer  
}
```

7 Worked example

This consolidated mock payload uses most of the contract in one grid:

Nested MC/Hist VaR + SVaR headers

Diverging Diff bar

\$MM badge

Action column (edit/delete/copy)

Global / AMER / EMEA total rows

Cell breach override

Master/detail

Two pinned bottom rows

String-format EQ T2 Limit

Grid-level heatmap

Saved sort / filter

● schemaVersion + gridMetaData

```
"schemaVersion": "2.0",
"gridMetaData": {
  "defaultColDef": { "sortable": true, "filter": true, "resizable":
true, "floatingFilter": true },
  "pagination": true,
  "paginationPageSize": 20,
  "rowSelection": "multiple",
  "getRowId": "id",
  "masterDetail": true,
  "pivotMode": false,
  "heatmap": {
    "scope": "grid", "min": -50, "max": 50, "centerline": 0,
    "positiveColor": "#2e7d32", "negativeColor": "#c62828"
  }
}
```

● columnDefs – nested headers

```
"columnDefs": [  
  { "headerName": "Desk", "field": "desk", "pinned": "left", "width":  
220, "colType": "StringType" },  
  { "headerName": "Unit", "field": "unit", "pinned": "left", "width":  
90, "colType": "StringType" },  
  { "headerName": "Monte Carlo VAR", "children": [  
    { "headerName": "COB", "field": "mcVarCob", "colType":  
"DoubleType" },  
    { "headerName": "PCOB", "field": "mcVarPcob", "colType":  
"DoubleType" },  
    { "headerName": "Diff", "field": "mcVarDiff", "colType":  
"DoubleType" } ] },  
  { "headerName": "Historical VAR", "children": [  
    { "headerName": "COB", "field": "histVarCob", "colType":  
"DoubleType" },  
    { "headerName": "PCOB", "field": "histVarPcob", "colType":  
"DoubleType" },  
    { "headerName": "Diff", "field": "histVarDiff", "colType":  
"DoubleType" } ] },  
  { "headerName": "Monte Carlo SVAR", "children": [  
    { "headerName": "COB", "field": "mcSvarCob", "colType":  
"DoubleType" },  
    { "headerName": "PCOB", "field": "mcSvarPcob", "colType":  
"DoubleType" },  
    { "headerName": "Diff", "field": "mcSvarDiff", "colType":  
"DoubleType" } ] },  
  { "headerName": "Historical SVAR", "children": [  
    { "headerName": "COB", "field": "histSvarCob", "colType":  
"DoubleType" },  
    { "headerName": "PCOB", "field": "histSvarPcob", "colType":  
"DoubleType" },  
    { "headerName": "Diff", "field": "histSvarDiff", "colType":  
"DoubleType" } ] },  
  { "headerName": "Actions", "field": "actions", "pinned": "right",  
"width": 120, "colType": "CustomType" }  
]
```

● columnMetadata

```
"columnMetadata": {  
  "desk": {  
    "tooltip": { "renderer": "customTooltip" },  
    "filter": "agSetColumnFilter",  
    "filterParams": { "filterType": "set", "values": ["NA", "EMEA"] }  
  },  
  "unit": { "type": "badge", "badge": { "static": true, "value":  
"$MM" } },  
  "mcVarCob": {  
    "sort": "desc", "sortIndex": 0,  
    "filter": "agNumberColumnFilter",  
    "filterParams": { "filterType": "number", "defaultOption":  
"greaterThan", "defaultValue": "0" },  
    "tooltip": { "renderer": "customTooltip" }  
  },  
  "mcVarDiff": { "type": "bar" },  
  "histVarDiff": { "type": "bar" },  
  "mcSvarDiff": { "type": "bar" },  
  "histSvarDiff": { "type": "bar" },  
  "actions": {  
    "type": "action",  
    "action": { "items": [  
      { "id": "edit", "icon": "edit", "handler": "onRowAction" },  
      { "id": "delete", "icon": "trash", "handler": "onRowAction" },  
      { "id": "copy", "icon": "copy", "handler": "onRowAction" } ]  
    }  
  }  
}
```

● **rowData** (raw values only)

```
"rowData": [  
  { "id": "total_global", "desk": "Global Total", "unit": "$MM",  
    "mcVarCob": -12.4, "mcVarPcob": -10.1, "mcVarDiff": -2.3,  
    "histVarCob": -14.0, "histVarPcob": -13.2, "histVarDiff": -0.8,  
    "mcSvarCob": -28.5, "mcSvarPcob": -27.9, "mcSvarDiff": -0.6,  
    "histSvarCob": -31.0, "histSvarPcob": -30.4, "histSvarDiff": -0.6  
  },  
  { "id": "total_amer", "desk": "AMER Total", "unit": "$MM",  
    "mcVarCob": -7.2, "mcVarPcob": -6.0, "mcVarDiff": -1.2, ... },  
  { "id": "row_1", "desk": "Derivatives Desk A", "unit": "$MM",  
    "mcVarCob": -3.1, "mcVarPcob": -2.6, "mcVarDiff": -0.5, ... },  
  { "id": "row_2", "desk": "Derivatives Desk B", "unit": "$MM",  
    "mcSvarCob": 2.0, "mcSvarPcob": 1.2, "mcSvarDiff": 0.8, ... },  
  { "id": "total_emea", "desk": "EMEA Total", "unit": "$MM", ... },  
  { "id": "row_3", "desk": "Financing Desk B", "unit": "$MM", ... }  
]
```

● rowMetadata + cellMetadata

```
"rowMetadata": {
  "total_global": { "rowType": "GRAND_TOTAL", "bold": true,
"bgColor": "#eceff1",
                    "excludeHeatMap": true, "selectable": false },
  "total_amer": { "rowType": "SUB_TOTAL", "bold": true, "bgColor":
"#f5f5f5",
                  "excludeHeatMap": true, "selectable": false },
  "total_emea": { "rowType": "SUB_TOTAL", "bold": true, "bgColor":
"#f5f5f5",
                  "excludeHeatMap": true, "selectable": false },
  "row_1": { "actions": { "edit": { "enabled": true } } },
  "row_2": { "actions": { "edit": { "enabled": true }, "delete": {
"enabled": true } } },
  "row_3": { "error": true, "actions": { "edit": { "enabled": true }
} }
},
"cellMetadata": {
  "row_2": { "mcSvarCob": { "bgColor": "#ffebee", "color": "#c62828",
"tooltip": "Breachd MC SVaR appetite by 1.8MM", "icon":
"warning" } },
  "row_3": { "mcVarCob": { "tooltip": "Full precision: -2.634812" }
}
}
```

● detailData + pinnedBottomRows

```
"detailData": {
  "row_2": {
    "columnDefs": [
      { "headerName": "Sub-Desk", "field": "subDesk", "colType":
"StringType" },
      { "headerName": "COB", "field": "cob", "colType": "DoubleType"
} ],
    "rowData": [
      { "id": "row_2_d1", "subDesk": "Rates", "cob": -1.2 },
      { "id": "row_2_d2", "subDesk": "FX", "cob": -2.9 } ]
    }
  },
  "pinnedBottomRows": [
    { "id": "pinned_grandTotal", "desk": "Grand Total", "unit": "$MM",
      "mcVarCob": -24.8, "mcVarPcob": -20.2, "mcVarDiff": -4.6, ... },
    { "id": "pinned_eqT2Limit", "desk": "EQ T2 Limit", "unit": "$MM",
      "mcVarCob": "-1500|150", "mcVarPcob": "-1500|150",
      "mcVarDiff": "",
      "histVarCob": "-1500|150", "histVarPcob": "-1500|150",
      "histVarDiff": "",
      "mcSvarCob": "-3000|300", "mcSvarPcob": "-3000|300",
      "mcSvarDiff": "",
      "histSvarCob": "-3000|300", "histSvarPcob":
"-3000|300", "histSvarDiff": "" }
  ]
}
// "min|max" strings are parsed by the UI into { min, max }; "" = no
limit.
```

8 Proto mapping appendix

Each JSON block maps to a proto message. `optional` is used wherever "absent" means something different from the zero value.

JSON BLOCK	PROTO	NOTES
<code>schemaVersion</code>	<code>string schema_version</code>	Plain; required.

gridMetadata	<code>GridMetadata</code>	Mostly optional bool/int32; heatmap → HeatmapConfig.
columnDefs	<code>repeated ColumnDef</code>	Recursive: ColumnDef.children.
columnMetadata	<code>map<string, ColumnMeta></code>	oneof config keyed by type.
rowData	<code>repeated google.protobuf.Struct</code>	Raw values; id required.
rowMetadata	<code>map<string, RowMeta></code>	Sparse — only rows that need it.
cellMetadata	<code>map<string, CellMetaRow></code>	CellMetaRow = map<string, CellMeta>.
detailData	<code>map<string, DetailRows></code>	Or detail_endpoint for lazy fetch.
pinnedTopRows	<code>repeated Struct</code>	Same shape as rowData.
pinnedBottomRows	<code>repeated Struct</code>	Totals and limit rows.

OPTIONAL vs PLAIN

- **optional:** booleans and numbers where unset means "use default" (sortable, width, min/max).
- **plain:** required keys and strings where empty is the same as unset (field, headerName).
- Free-form values (rowData cells) use `google.protobuf.Value`.

FUNCTION ESCAPE HATCHES

- `getRowId` → `rowIdField: "id";`
the UI builds the function.
- Detail / tree callbacks → `detailField`, `treePathField` or `detailEndpoint` for lazy fetch.
- Everything else uses registry names.